

Model Context Protocol (MCP)

A practical, text-based guide to connecting AI models with external tools, data, and services

1. What is MCP?

Model Context Protocol (MCP) is an open protocol designed to standardize how AI applications connect to external data sources, tools, and services. Instead of building a separate custom integration for every model, agent framework, database, API, or application, an MCP-compatible client can communicate with MCP servers through a common interface.

The key idea is simple: **the model reasons, while external systems provide capabilities and context**. MCP defines a structured way for an AI application to discover and use those capabilities.

Why MCP is useful

- **Standardization:** one protocol can expose tools and context to many compatible AI applications.
- **Reusability:** an MCP server can encapsulate an integration once and make it available to multiple clients.
- **Separation of concerns:** application logic, model reasoning, and external-system integrations can remain separate.
- **Tool discovery:** clients can learn which tools are available and what inputs they require.
- **Security boundaries:** the host can control which servers, tools, resources, and permissions are exposed.

2. Core Architecture

A typical MCP setup has three major roles:

Component	Responsibility
MCP Host	The AI application that coordinates the model, user interaction, permissions, and MCP connections.
MCP Client	The protocol-side component inside the host that maintains a connection to an MCP server and exchanges MCP messages.
MCP Server	A program that exposes capabilities such as tools, resources, and prompts to an MCP client.

A useful mental model is:

User → AI Host → Model + MCP Client → MCP Server → External System

For example, an AI coding assistant may connect to an MCP server that exposes repository files, issue trackers, documentation, or database operations. The model does not need a bespoke integration for each system; it can interact with the capabilities exposed through the MCP interface.

3. What MCP Servers Can Expose

- **Tools:** executable functions that allow the AI application to perform actions, such as querying a database, creating a ticket, calling an API, or running an approved operation.
- **Resources:** contextual information that can be retrieved, such as documents, files, records, schemas, or other application data.
- **Prompts:** reusable prompt templates or interaction patterns that a server can make available to clients.

The distinction matters: a resource primarily supplies context, a tool performs an operation, and a prompt provides a reusable instruction pattern.

4. MCP in an Agentic AI System

MCP is especially valuable for agentic systems because agents frequently need to move beyond generating text. An agent may need to inspect information, choose an action, invoke a tool, observe the result, and continue reasoning. MCP provides a standardized capability layer for that interaction.

A simplified loop is: **discover capability** → **decide whether to use it** → **call tool/resource** → **receive result** → **reason over result** → **continue or respond**.

MCP — Protocol Concepts and Workflow

5. How an MCP Interaction Works

At a high level, the interaction follows a request/response pattern. The MCP client establishes a session with an MCP server, negotiates supported capabilities, and can then request available tools or resources. When the model decides that an external capability is needed, the host asks the client to invoke the appropriate MCP operation.

A simplified sequence looks like this:

- **1. Connection:** the host creates or obtains an MCP client connection to a server.
- **2. Initialization:** client and server establish protocol/session information and negotiate capabilities.
- **3. Discovery:** the client can learn about available tools, resources, and prompts.
- **4. Model decision:** the AI model determines that external information or an action is required.
- **5. Invocation:** the host asks the MCP client to call a selected tool or retrieve a resource.
- **6. Result:** the server returns structured content or an error.
- **7. Continued reasoning:** the model uses the result as context and decides what to do next.

6. Transport and Communication

MCP defines the protocol semantics and message structure, while deployments can use different communication mechanisms depending on where the server runs. A server may run locally as a process launched by the host, or it may be reached through a networked transport.

The important architectural point is that **the protocol interface remains consistent even when the underlying integration changes**. This allows an MCP client to treat many integrations as standardized capability providers.

7. Tools: Turning APIs into Agent Capabilities

An MCP tool is typically described with a name, description, and input schema. The schema tells the client and model what arguments the tool accepts. A well-designed tool should have a narrow, explicit purpose.

Example conceptual tool:

Tool name: search_customer_cases

Purpose: Search support cases using structured filters.

Inputs: customer_id, date_range, status, limit.

Output: structured case records or an error.

Good tool design is important for agents. Tools should have clear descriptions, predictable inputs and outputs, appropriate validation, and minimal permissions. A tool that exposes an unrestricted database or arbitrary shell command is much harder to govern safely than a narrowly scoped business operation.

8. Resources: Supplying Context

Resources are useful when an application needs to make information available to the model without treating every piece of information as an executable action. Examples include project documentation, database schemas, configuration information, product manuals, or files.

Resources can therefore support retrieval-heavy workflows. An agent might first retrieve a relevant document or schema, then use a tool to perform an action based on that context.

9. Prompts: Reusable Interaction Patterns

MCP prompts allow servers to expose reusable prompt templates or workflows. This can help teams centralize domain-specific instructions instead of duplicating them across many client applications.

For example, an internal engineering MCP server could provide a prompt template for reviewing a production incident, while another server could expose a standardized prompt for analyzing customer support cases.

10. Structured Results and Errors

Agent systems need more than successful responses. Tools can fail because of invalid input, missing permissions, unavailable services, rate limits, or downstream errors. MCP interactions therefore need to preserve enough structure for the host to distinguish a useful result from an operational failure.

This is important in multi-step agent loops: **the model should receive an understandable result or error, rather than being left to infer what happened from an ambiguous response.**

MCP — Production Design, Security, and Examples

11. MCP vs. Traditional Integrations

Traditional approach	MCP-oriented approach
Each AI application implements custom integrations.	A common protocol provides a reusable integration boundary.
Tool definitions can be tightly coupled to one framework.	Capabilities can be exposed through a standardized interface.
Changing AI applications may require rebuilding integrations.	Compatible clients can potentially reuse existing MCP servers.
Permissions are often implemented independently per integration.	The host/server boundary can be used as an explicit governance point.

12. Security Considerations

MCP does not automatically make an agent secure. It creates an interface through which capabilities can be exposed, so security must be designed around that interface.

- **Least privilege:** expose only the tools and data the application actually needs.
- **Input validation:** validate tool arguments before performing downstream operations.
- **Authentication and authorization:** protect sensitive servers and enforce user/service permissions.
- **Human approval:** consider approval gates for destructive, financial, or high-impact operations.
- **Data minimization:** avoid returning unnecessary sensitive information to the model.
- **Auditability:** record tool calls, identities, outcomes, and relevant metadata according to organizational policy.
- **Server trust:** treat third-party MCP servers as software integrations that require security review.

13. Example: Customer Support Agent

Imagine an enterprise support agent connected to an MCP server. The server exposes three tools: **search_cases**, **get_product_documentation**, and **create_escalation**.

A customer asks why a particular server alert keeps recurring. The agent can retrieve related cases, consult product documentation, identify the likely resolution, and—if appropriate—create an escalation. The model handles reasoning and conversation; the MCP server provides controlled access to enterprise capabilities.

This pattern can be extended to RAG systems, coding assistants, analytics agents, IT operations agents, and multi-agent architectures.

14. MCP in a Modern AI Architecture

Application / Agent

↓

LLM reasoning + orchestration

↓

MCP client layer

↓

MCP servers

↓

Databases • APIs • files • SaaS systems • internal services

This architecture helps separate **reasoning** from **capability access**. The model can remain interchangeable, while integrations can be maintained as independent capability providers.

15. Key Takeaways

- MCP is a protocol for standardizing how AI applications connect to external capabilities and context.
- Its central architecture separates the host, MCP client, and MCP server responsibilities.
- Servers can expose tools, resources, and prompts.
- Tools enable actions; resources provide context; prompts provide reusable interaction patterns.
- MCP is particularly useful for agentic AI because agents need reliable access to external systems during reasoning loops.
- MCP is an integration standard, not a complete security solution. Permissions, validation, authentication, approvals, and auditing remain essential.

In short: **MCP can be viewed as a standardized capability layer between AI applications and the systems they need to interact with.**

Note: MCP is an evolving open protocol. For implementation work, always consult the current official specification and SDK documentation for the exact protocol version, transports, authorization mechanisms, and capability details.